hello i want you to explain the folloging code word by word ,line by line for a beginawhat teach code does and etc

I'd be happy to explain code line by line! However, I notice you didn't include the actual code in your message.

Could you please **paste the code** you'd like me to explain? Once you share it, I'll break it down:

- **Word by word** – explaining what each keyword/variable/function name means
- **Line by line** – walking through what happens at each step
- **Overall purpose** – what the program is designed to do
- **Beginner-friendly** – no jargon without explanation

Please paste the code (using a code block if possible, like `code here`), and let me know:

1. What **programming language** it's in (or I can guess if you're unsure)
2. Any **specific parts** that confuse you most

Go ahead and share it! 🚀

ai.py
PY 1.48KB

inventory.py
PY 2.22KB

mpesa.py
PY 2.5KB

whatsapp.py
PY 9.01KB

Complete Beginner-Friendly Walkthrough

I'll explain each file, what it does, and go through it line by line. Take your time — this is a lot of code, so I'll break it into digestible chunks.

🌍 Big Picture: What is this project?

This is a **backend for a Kenyan phone store** (called "X-UK") that does three main things:

1. **AI chat** — talks to an LLM (a large language model like ChatGPT) via OpenRouter
2. **Inventory management** — tracks product stock when orders are placed/cancelled
3. **WhatsApp bot + M-Pesa payments** — automatically chats with customers on WhatsApp, qualifies them as sales leads, and processes mobile-money payments

Let me explain each file.

📄 File 1: `ai.py` — The AI Helper

This file is the **single place** where the app talks to the AI provider (OpenRouter). Other parts of the app call it.

```
python

import os
from typing import Any, Dict, List, Optional
```

- `import os` — gives us access to environment variables (secret settings stored outside the code, like API keys)
- `from typing import ...` — imports **type hints** (labels that tell you what kind of data a variable should be)
  - `Any` = any type
  - `Dict` = dictionary (key/value pairs, like `{"name": "John"}`)
  - `List` = list (like `[1, 2, 3]`)
  - `Optional` = "this can be that type OR None"

```
python

import httpx
from fastapi import HTTPException
```

- `httpx` — a library for making HTTP requests (talking to web APIs)
- `HTTPException` — a FastAPI error class. When raised, FastAPI returns an error response to the caller automatically.

```python
OPENROUTER_API_KEY = os.environ.get("OPENROUTER_API_KEY")
OPENROUTER_URL = "https://openrouter.ai/api/v1/chat/completions"
DEFAULT_MODEL = "meta-llama/llama-3.1-8b-instruct"
```

- Line 1: reads the API key from an environment variable (a secret password for OpenRouter). If it's not set, this is `None`.
- Line 2: the URL we POST to when we want AI responses.
- Line 3: default AI model = Meta's Llama 3.1 8B Instruct.

```python
async def chat_completion(
    messages: List[Dict[str, str]],
    model: str = DEFAULT_MODEL,
    temperature: float = 0.7,
    max_tokens: int = 600,
    response_format: Optional[Dict[str, Any]] = None,
) -> str:
```

This defines an **async function** called `chat_completion`.

- `async` = it can pause while waiting for network calls (so the server can handle other requests meanwhile)
- Parameters:
  - `messages` — a list of chat messages, each with a `role` ("user"/"system"/"assistant") and `content`
  - `model` — which AI model (defaults to the Llama one)
  - `temperature` — randomness. 0 = very predictable, 1 = very creative. 0.7 is balanced.
  - `max_tokens` — maximum length of the reply
  - `response_format` — optional; can force the AI to output JSON
- `-> str` — this function returns a string

python

```
    """Call OpenRouter and return the assistant's text content.
    ...
    """
```

A **docstring** — documentation explaining what the function does. Not executed.

python

```
    if not OPENROUTER_API_KEY:
        raise HTTPException(status_code=500, detail="AI is not configured")
```

If no API key is set, throw a 500 error ("server misconfigured") and stop.

python

```
    payload: Dict[str, Any] = {
        "model": model,
        "messages": messages,
        "temperature": temperature,
        "max_tokens": max_tokens,
    }
    if response_format:
        payload["response_format"] = response_format
```

Builds the JSON body we'll send to OpenRouter. If the caller passed `response_format`, add it.

python

```
    async with httpx.AsyncClient() as client:
        response = await client.post(
            OPENROUTER_URL,
            headers={"Authorization": f"Bearer {OPENROUTER_API_KEY}"},
            json=payload,
            timeout=60.0,
        )
```

- Opens an async HTTP client
- POSTs our payload to OpenRouter

- POSTs our payload to OpenRouter
- `Authorization: Bearer <key>` — the standard way to authenticate with an API
- `timeout=60.0` — give up after 60 seconds

```
python

    if response.status_code != 200:
        raise HTTPException(status_code=502, detail=f"AI provider error: {response.tex
t}")

    data = response.json()
    return data["choices"][0]["message"]["content"]
```

- If the provider didn't return 200 (OK), raise a 502 error with the provider's message
- Otherwise, parse the JSON response and dig into it:
  - `choices[0]` = first completion option
  - `message.content` = the actual text of the reply
- Return that text.

📄 File 2: `inventory.py` — Stock Management

Handles product stock when orders are placed/cancelled.

```
python

from typing import Any, Dict

from app.database import supabase

STOCK_HELD_STATUSES = {"Pending PIN", "Processing"}
```

- `supabase` — a client for Supabase (a hosted Postgres database). It has a fluent API like `supabase.table("products").select(...)`
- `STOCK_HELD_STATUSES` — a **set** of order statuses where stock is currently reserved/held (not yet sold)

Function 1: `restore_order_stock`

```python
def restore_order_stock(order: Dict[str, Any]) -> None:
    """Increment product stock back for every line item on an order."""
    items = order.get("items")
    if not isinstance(items, list):
        return
```

- Takes an order (a dict), gets its `items` field
- If `items` isn't a list, bail out (defensive coding — don't crash if data is malformed)

```python
    for item in items:
        if not isinstance(item, dict):
            continue

        product_id = item.get("id") or item.get("product_id")
        if not product_id:
            continue

        try:
            quantity = int(item.get("qty"))
        except (TypeError, ValueError):
            continue

        if quantity <= 0:
            continue
```

For each item:

- Skip if it's not a dict
- Get the product ID (try `id`, fall back to `product_id`)
- Skip if no product ID
- Try to convert `qty` to an integer; skip if it fails
- Skip non-positive quantities

```python
        res = (
            supabase.table("products")
            .select("stock_quantity")
            .eq("id", product_id)
            .execute()
        )
        if not res.data or not isinstance(res.data[0], dict):
            continue
```

- Read the current stock for that product
- `eq("id", product_id)` = "where id equals product_id"
- Skip if no data came back

```python
        try:
            current_stock = int(res.data[0].get("stock_quantity") or 0)
        except (TypeError, ValueError):
            current_stock = 0

        supabase.table("products").update(
            {"stock_quantity": current_stock + quantity}
        ).eq("id", product_id).execute()
```

- Get current stock as int (fall back to 0 on error)
- Add the quantity back and update the database — this "returns" the reserved stock

Function 2: `release_stock_once`

```python
def release_stock_once(order: Dict[str, Any], new_status: str = "Payment Failed") -> bo
ol:
    """Atomically move a still-held order to a failed/cancelled state and restore stoc
k.
    ...
    """
```

"Atomically" means: do it in one safe operation so that if two requests arrive at the same time (e.g., duplicate callbacks from M-Pesa), only one actually performs the change.

```
python

    if not isinstance(order, dict):
        return False

    current_status = order.get("status")
    if current_status not in STOCK_HELD_STATUSES:
        return False

    order_id = order.get("id")
    if not order_id:
        return False
```

- Sanity checks: must be a dict, must have a status that's currently holding stock, must have an ID

```
python

    res = (
        supabase.table("orders")
        .update({"status": new_status})
        .eq("id", order_id)
        .eq("status", current_status)
        .execute()
    )
    if not res.data:
        return False
```

The magic line: `.eq("status", current_status)` means "only update if status is *still* the old one." If another request already changed it, this update affects 0 rows → `res.data` is empty → return `False`.

```
python

    restore_order_stock(order)
    return True
```

We won the race — restore stock and tell the caller we did the release.

📄 File 3: `mpesa.py` — M-Pesa Payment Callbacks

M-Pesa is Kenya's mobile money system. When a customer pays via "STK Push" (a prompt on their phone), M-Pesa sends a callback to this server. This file handles that callback.

```
python
```

```python
from typing import Any, Dict

from app.database import supabase
from app.services.inventory import release_stock_once
from app.utils.email import send_receipt_email
```

Imports what we need: DB, the stock-release helper, and an email sender.

```
python
```

```python
def _safe_total(value: Any) -> float:
    try:
        if isinstance(value, bool):
            return 0.0
        return float(value)
    except (TypeError, ValueError):
        return 0.0
```

A helper: convert anything to a float safely. Note: in Python, `bool` is a subclass of `int`, so `True` → `1.0` — we explicitly block that.

```
python
```

```python
def process_stk_callback(payload: Dict[str, Any]) -> None:
    """Single source of truth for M-Pesa STK Push callbacks. ..."""
    try:
        body = payload.get("Body", {}).get("stkCallback", {})
        checkout_request_id = body.get("CheckoutRequestID")
        result_code = body.get("ResultCode")
```

```
        result_code = body.get( ResultCode )

        if not checkout_request_id:
            return
```

- Dig into the nested M-Pesa payload structure to find the checkout ID and result code
- If there's no checkout ID, exit (nothing to do)

python

```
        order_res = (
            supabase.table("orders")
            .select("*")
            .eq("checkout_request_id", checkout_request_id)
            .execute()
        )
        order = order_res.data[0] if order_res.data and isinstance(order_res.data[0], d
ict) else None
```

Look up our order by the checkout ID.

python

```
        if result_code == 0:
```

`0` means payment succeeded.

python

```
            meta_items = body.get("CallbackMetadata", {}).get("Item", [])
            receipt = "UNKNOWN"
            for item in meta_items:
                if item.get("Name") == "MpesaReceiptNumber":
                    receipt = item.get("Value")
```

Extract the M-Pesa receipt number from the metadata.

python

```
            if order and order.get("status") != "Paid":
                supabase.table("orders").update(
```

```
                {"status": "Paid", "receipt_number": receipt}
            ).eq("checkout_request_id", checkout_request_id).execute()

            customer_email = order.get("customer_email")
            if isinstance(customer_email, str) and customer_email:
                order_items = order.get("items", [])
                if not isinstance(order_items, list):
                    order_items = []

                send_receipt_email(
                    customer_email,
                    str(order.get("order_reference", "")),
                    _safe_total(order.get("total_amount")),
                    order_items,
                )
```

- Mark order as `Paid` (only if not already paid — idempotent)
- Email the customer a receipt

python

```
        else:
            new_status = "Cancelled" if result_code == 1032 else "Payment Failed"
            if order and release_stock_once(order, new_status):
                print(f"Released stock for order {checkout_request_id} ({new_status})")
```

If payment failed:

- `1032` = user cancelled → status `Cancelled`
- Any other failure → `Payment Failed`
- Release the held stock **exactly once** (the `release_stock_once` handles that)

python

```
    except Exception as e:
        print(f"CRITICAL ERROR processing M-Pesa callback: {e}")
```

Catch everything so a bug doesn't crash the whole server — just log it.

📄 File 4: `whatsapp.py` — The WhatsApp Sales Bot

This is the biggest file. It handles inbound WhatsApp messages, runs them through the AI, saves lead info, and notifies the owner when a hot lead appears.

```
python

import hashlib
import hmac
import json
import os
from datetime import datetime, timezone
from typing import Any, Dict, List, Optional

import httpx

from app.database import supabase
from app.services.ai import chat_completion
```

- `hashlib` / `hmac` — for verifying that webhook requests really came from Meta (Facebook/WhatsApp)
- `json` — for parsing/creating JSON
- `datetime, timezone` — for timestamps
- `chat_completion` — the AI helper from `ai.py`

```
python

WHATSAPP_TOKEN = os.environ.get("WHATSAPP_TOKEN")
WHATSAPP_PHONE_ID = os.environ.get("WHATSAPP_PHONE_ID")
OWNER_WHATSAPP_NUMBER = os.environ.get("OWNER_WHATSAPP_NUMBER")
GRAPH_VERSION = os.environ.get("WHATSAPP_GRAPH_VERSION", "v20.0")

HISTORY_LIMIT = 8
HOT_THRESHOLD = int(os.environ.get("LEAD_HOT_THRESHOLD", "70"))
WARM_THRESHOLD = int(os.environ.get("LEAD_WARM_THRESHOLD", "40"))
```

Config: API credentials, how many past messages to remember, score thresholds for lead quality.

```
python

TRIAGE_SYSTEM_PROMPT = """You are the friendly sales assistant for X-UK, ...
{
  "reply": "...",
  "intent": "browsing|price|purchase|support|other",
  ...
  "score": 0,
  "stage": "new|warm|hot|cold"
}
..."""
```

This is the **system prompt** given to the AI on every conversation. It tells the AI:

- Who it is
- To ask ONE question at a time
- To always respond with a specific JSON structure
- What each field means

Section: Webhook Security

```
python

def verify_signature(raw_body: bytes, signature_header: Optional[str]) -> bool:
    """Validate Meta's `X-Hub-Signature-256` header against the app secret. ..."""
    app_secret = os.environ.get("WHATSAPP_APP_SECRET")
    if not app_secret:
        print("[whatsapp] WHATSAPP_APP_SECRET not set; skipping signature check")
        return True
```

Meta (WhatsApp's parent) signs every webhook request with your app secret. This function verifies the signature to prove the request is genuine — otherwise anyone could post fake messages.

```
python

    if not signature_header or not signature_header.startswith("sha256="):
        return False
```

```
    expected = hmac.new(app_secret.encode(), raw_body, hashlib.sha256).hexdigest()
    provided = signature_header.split("=", 1)[1]
    return hmac.compare_digest(expected, provided)
```

- If header is missing or malformed, reject
- Recompute the HMAC-SHA256 using our secret
- Compare in **constant time** (`compare_digest`) — this prevents attackers from guessing the signature one character at a time via timing

Section: WhatsApp Cloud API

python

```
async def send_whatsapp_message(to: str, body: str) -> bool:
    if not WHATSAPP_TOKEN or not WHATSAPP_PHONE_ID:
        print("[whatsapp] not configured; skipping outbound send")
        return False
```

Send a message via Meta's Graph API. If credentials aren't configured, skip.

python

```
    url = f"https://graph.facebook.com/{GRAPH_VERSION}/{WHATSAPP_PHONE_ID}/messages"
    payload = {
        "messaging_product": "whatsapp",
        "to": to,
        "type": "text",
        "text": {"body": body},
    }
    async with httpx.AsyncClient() as client:
        res = await client.post(
            url,
            headers={"Authorization": f"Bearer {WHATSAPP_TOKEN}"},
            json=payload,
            timeout=30.0,
        )
    if res.status_code >= 300:
        print(f"[whatsapp] send failed {res.status_code}: {res.text}")
        return False
    return True
```

POST the message to Meta. Return `True` on success, `False` on failure.

```python
def extract_inbound_message(payload: Dict[str, Any]) -> Optional[Dict[str, Any]]:
    """Pull the first text message out of a Meta webhook payload."""
    try:
        value = payload["entry"][0]["changes"][0]["value"]
        messages = value.get("messages") or []
        if not messages:
            return None
        message = messages[0]

        name = None
        contacts = value.get("contacts") or []
        if contacts:
            name = (contacts[0].get("profile") or {}).get("name")

        text = ""
        if message.get("type") == "text":
            text = (message.get("text") or {}).get("body", "")

        return {
            "from": message.get("from"),
            "text": text,
            "id": message.get("id"),
            "name": name,
        }
    except (KeyError, IndexError, TypeError):
        return None
```

Meta's webhook payload is deeply nested. This navigates the structure and pulls out the sender's phone, message text, message ID, and name. If anything's missing or malformed, return `None`.

Section: Lead Persistence

```python
def get_or_create_lead(phone: str, name: Optional[str] = None) -> Dict[str, Any]:
```

```
    res = supabase.table("leads").select("*").eq("phone", phone).execute()
    if res.data:
        lead = res.data[0]
        if name and not lead.get("name"):
            supabase.table("leads").update({"name": name}).eq("id", lead["id"]).execute
()
            lead["name"] = name
        return lead

    inserted = (
        supabase.table("leads")
        .insert({"phone": phone, "name": name, "source": "whatsapp"})
        .execute()
    )
    return inserted.data[0]
```

Look up the lead by phone. If found, optionally fill in the name and return it. If not, insert a new lead and return it.

python

```
def log_message(lead_id: str, direction: str, body: str, wa_message_id: Optional[str] =
None) -> None:
    supabase.table("whatsapp_messages").insert({
        "lead_id": lead_id,
        "direction": direction,
        "body": body,
        "wa_message_id": wa_message_id,
    }).execute()
```

Save each message (inbound or outbound) so we have a full conversation history.

python

```
def fetch_history(lead_id: str, limit: int = HISTORY_LIMIT) -> List[Dict[str, Any]]:
    res = (
        supabase.table("whatsapp_messages")
        .select("direction, body")
        .eq("lead_id", lead_id)
        .order("created_at", desc=True)
        .limit(limit)
        .execute()
```

```
    )
    return list(reversed(res.data or []))
```

Get the last N messages (newest first from DB), then reverse them so oldest is first (correct chat order for the AI).

python

```
def _status_from_score(score: int) -> str:
    if score >= HOT_THRESHOLD:
        return "hot"
    if score >= WARM_THRESHOLD:
        return "warm"
    return "cold"
```

Score → status label mapping. The `_` prefix is a Python convention meaning "internal/private helper."

python

```
def _apply_meta(lead_id: str, meta: Dict[str, Any]) -> Dict[str, Any]:
    update: Dict[str, Any] = {
        "last_message_at": datetime.now(timezone.utc).isoformat(),
    }

    try:
        score = int(meta.get("score"))
        score = max(0, min(100, score))
        update["score"] = score
        update["status"] = _status_from_score(score)
    except (TypeError, ValueError):
        pass

    for field in ("interest", "timeline", "location", "budget"):
        value = meta.get(field)
        if value not in (None, "", "null"):
            update[field] = value

    supabase.table("leads").update(update).eq("id", lead_id).execute()
    return update
```

Apply the AI's extracted info (score, intent, budget, etc.) to the lead row.

- Always update `last_message_at`
- Try to clamp the score to 0–100 and derive the status
- Copy over non-empty fields
- Save and return what we updated

```python
def _parse_model_output(raw: str):
    text = (raw or "").strip()
    if text.startswith("```"):
        text = text.strip("`")
        if text.lower().startswith("json"):
            text = text[4:].strip()

    start, end = text.find("{"), text.rfind("}")
    if start != -1 and end != -1:
        try:
            data = json.loads(text[start:end + 1])
            reply = str(data.get("reply") or "").strip()
            if reply:
                return reply, data
        except json.JSONDecodeError:
            pass
    return text, {}
```

The AI is supposed to return pure JSON, but sometimes it wraps it in markdown code fences (```` ```json ... ``` ````). This strips those off and extracts the JSON object. Returns `(reply_text, full_metadata_dict)` . If parsing fails, use the raw text as the reply and empty metadata.

```python
async def _notify_owner(lead: Dict[str, Any], meta: Dict[str, Any]) -> None:
    if not OWNER_WHATSAPP_NUMBER:
        return
    summary = (
        f"🔥 HOT LEAD\n"
        f"Name: {lead.get('name') or 'Unknown'}\n"
        f"Phone: {lead.get('phone')}\n"
```

```
        f"Phone: {lead.get('phone')}\n"
        f"Interest: {meta.get('interest') or '-'}\n"
        f"Budget: {meta.get('budget') or '-'}\n"
        f"Score: {meta.get('score')}\n"
        f"Timeline: {meta.get('timeline') or '-'}"
    )
    await send_whatsapp_message(OWNER_WHATSAPP_NUMBER, summary)
```

When a lead turns "hot," send a WhatsApp alert to the store owner.

Section: Main Entry Point

python

```
async def handle_inbound_message(payload: Dict[str, Any]) -> None:
    try:
        message = extract_inbound_message(payload)
        if not message or not message.get("from"):
            return
```

Main function that processes an incoming WhatsApp message.

python

```
        phone = message["from"]
        text = message.get("text") or ""
        lead = get_or_create_lead(phone, message.get("name"))
        log_message(lead["id"], "inbound", text, message.get("id"))
```

- Extract phone and text
- Find or create the lead
- Log the inbound message

python

```
        history = fetch_history(lead["id"])
        messages = [{"role": "system", "content": TRIAGE_SYSTEM_PROMPT}]
        for row in history:
            messages.append({
                "role": "user" if row.get("direction") == "inbound" else "assistant",
```

```
            "content": row.get("body", ""),
        })
```

Build the conversation array for the AI:

- Start with the system prompt
- Then each past message with the correct role (`inbound` → user, `outbound` → assistant)

```
python

        raw = await chat_completion(messages, temperature=0.4, max_tokens=500)
        reply, meta = _parse_model_output(raw)
```

Ask the AI. Low temperature (0.4) = more consistent JSON output. Parse the reply and metadata.

```
python

        log_message(lead["id"], "outbound", reply)

        was_hot = lead.get("status") == "hot"
        updated = _apply_meta(lead["id"], meta)

        await send_whatsapp_message(phone, reply)

        if updated.get("status") == "hot" and not was_hot:
            await _notify_owner(lead, meta)
    except Exception as e:  # background task: never crash the worker
        print(f"[whatsapp] handle_inbound_message error: {e}")
```

- Log our reply
- Remember old status
- Update lead with new score/metadata
- Send the reply on WhatsApp
- If they just became hot (weren't hot before), alert the owner
- Catch-all so a bug never crashes the background worker

🧠 Key Concepts You Just Saw

Concept	What it means
Environment variables	Secrets stored outside code (`os.environ.get(...)`)
Type hints	Labels like `Dict[str, Any]` saying what types to expect
`async` / `await`	Non-blocking I/O — the server can do other work while waiting for the network
Idempotency	Doing something twice has the same effect as once (`release_stock_once`)
Race condition	Two requests hitting at once. The `.eq("status", current_status)` trick prevents double-processing
HMAC signature	A way to prove a webhook really came from Meta
Supabase fluent API	`.table().select().eq().execute()` chain
Defensive coding	`isinstance()` , `try/except` — never trust outside data

🔄 How the Files Work Together

```
text

Customer sends WhatsApp msg
        ↓
whatsapp.py: verify signature → extract message → save lead & msg
        ↓
whatsapp.py: build conversation → ai.py (chat_completion)
        ↓
OpenRouter AI responds with JSON (reply + score)
        ↓
```

```
whatsapp.py: parse → update lead → send reply → maybe notify owner

Meanwhile, customer pays via M-Pesa:
M-Pesa → mpesa.py: process_stk_callback
        ↓
On success: mark Paid + email receipt
On failure: inventory.py: release_stock_once → restore stock
```

Would you like me to go deeper on any specific part — like the **HMAC signature verification**, the **Supabase query syntax**, or the **AI prompt engineering**?

yes

Deep Dive: The Three Trickiest Parts

I'll go deep on each of the three you mentioned, with lots of examples and beginner-friendly explanations.

🔐 Part 1: HMAC Signature Verification

This is one of the most important security concepts in webhooks. Let me build it up from scratch.

The Problem It Solves

Your server has a public URL like `https://x-uk.com/webhook/whatsapp`. **Anyone on the internet** can POST to it. How do you know a request really came from Meta (WhatsApp) and not from a scammer pretending to be Meta?

You can't just check the IP address (attackers can spoof it). You can't rely on a password in the URL (it could leak). So Meta uses **HMAC**.

What is HMAC?

HMAC = **H**ash-based **M**essage **A**uthentication **C**ode.

Think of it like this:

1. You and Meta share a **secret key** (only the two of you know it)
2. Meta takes the message body + the secret and runs a math formula → gets a **signature** (a long string of letters/numbers)
3. Meta sends you both the message AND the signature
4. You take the message + your copy of the secret, run the SAME formula
5. If your signature matches Meta's signature → the message is genuine (only someone with the secret could produce it)

What is a Hash?

A **hash** is a one-way function. You put data in, you get a fixed-length string out:

```
python

"hello"    → "2cf24dba5fb0a30e26e83b2ac5b9e29e..."
"hello!"   → "ce06092fb948d9ffac7d1a376e404b26..."  # totally different!
```

Key properties:

- **Same input → same output** (deterministic)
- **Slightly different input → completely different output**
- **Can't reverse it** (you can't get "hello" back from the hash)

SHA-256 is a popular hash that outputs 64 hex characters.

HMAC vs Plain Hash

Why not just hash the message? Because an attacker could compute the hash too! HMAC mixes in a **secret key** so only someone with the key can produce the right signature.

```
text
```

```
signature = HMAC_SHA256(secret_key, message_body)
```

Now Back to the Code

```
python

import hashlib
import hmac
```

```
python

def verify_signature(raw_body: bytes, signature_header: Optional[str]) -> bool:
```

- `raw_body: bytes` — the **raw** request body, as raw bytes. This is critical: you must use the EXACT bytes Meta signed, before any parsing/whitespace changes.
- `signature_header` — the value of the `X-Hub-Signature-256` header Meta sent, which looks like `"sha256=abcdef1234..."`

```
python

    app_secret = os.environ.get("WHATSAPP_APP_SECRET")
    if not app_secret:
        print("[whatsapp] WHATSAPP_APP_SECRET not set; skipping signature check")
        return True
```

If the secret isn't configured, we skip the check. **This is fine for local dev** but in production this would be a security hole — anyone could send fake messages. The print statement warns you.

```
python

    if not signature_header or not signature_header.startswith("sha256="):
        return False
```

Meta always sends signatures in the format `sha256=<hexdigest>`. If the header is missing or malformed, reject.

```python
    expected = hmac.new(app_secret.encode(), raw_body, hashlib.sha256).hexdigest()
```

Let's unpack this line:

- `app_secret.encode()` — converts the secret string to bytes (HMAC requires bytes)
- `raw_body` — the message bytes (already bytes)
- `hashlib.sha256` — the hash algorithm to use
- `.hexdigest()` — returns the result as a hex string (like `"a3f5c..."`)

So `expected` is **what the signature SHOULD be** if the message genuinely came from Meta with our secret.

```python
    provided = signature_header.split("=", 1)[1]
```

`signature_header` is `"sha256=abcdef123..."`. `.split("=", 1)` splits at the FIRST `=` into `["sha256", "abcdef123..."]`. We take index `[1]` = the actual signature.

```python
    return hmac.compare_digest(expected, provided)
```

Why `compare_digest` and not `==`?

This is subtle but important. A naive comparison like `expected == provided` **short-circuits** — Python compares character by character and stops at the first mismatch.

That means comparing two strings that differ at position 1 is **faster** than comparing strings that match up to position 50. An attacker could measure response times to learn the correct signature **one character at a time**! (This is called a **timing attack**.)

`hmac.compare_digest` **always takes the same time** regardless of where the mismatch is. Problem solved.

Attack Scenario Walk-Through

Without signature verification:

```
text

Attacker: POST /webhook/whatsapp {"messages": [{"from": "attacker", "text": "give me a
refund"}]}
Your server: "OK, trusted source!" → processes fake message
```

With signature verification:

```
text

Attacker: POST /webhook/whatsapp {"messages": [...]}
           (no valid signature — they don't know the secret)
Your server: verify_signature(...) → False → reject with 403
```

🗄️ Part 2: Supabase Query Syntax

Supabase is a hosted PostgreSQL database. Its Python client uses a **fluent/chained** API. Let me break down each piece.

The Basic Shape

Every query follows this pattern:

```
python

supabase.table("TABLE_NAME").OPERATION().FILTERS().execute()
```

You're **chaining method calls** — each returns something you can chain the next call onto.

Real Examples from the Code

Example 1: Read a single column

```python
res = (
    supabase.table("products")
    .select("stock_quantity")
    .eq("id", product_id)
    .execute()
)
```

Translated to SQL:

```sql
SELECT stock_quantity FROM products WHERE id = 'product_id';
```

Breakdown:

- `supabase.table("products")` — target the `products` table
- `.select("stock_quantity")` — which columns? Just `stock_quantity`
- `.eq("id", product_id)` — WHERE id = product_id
- `.execute()` — actually run it!

The result `res` has a `.data` attribute holding a list of dicts:

```python
res.data  # → [{"stock_quantity": 5}]
```

Example 2: Read everything

```python
res = supabase.table("orders").select("*").eq("checkout_request_id", checkout_request_id).execute()
```

- `.select("*")` — all columns (like `SELECT *`)

- `.eq("checkout_request_id", checkout_request_id)` — the WHERE clause

Example 3: Update

```python
supabase.table("orders").update(
    {"status": "Paid", "receipt_number": receipt}
).eq("checkout_request_id", checkout_request_id).execute()
```

SQL:

```sql
UPDATE orders SET status = 'Paid', receipt_number = '...' WHERE checkout_request_id =
'...';
```

- `.update({"col": val, ...})` — what to set
- `.eq(...)` — WHERE clause
- `.execute()` — run

Example 4: Insert

```python
inserted = (
    supabase.table("leads")
    .insert({"phone": phone, "name": name, "source": "whatsapp"})
    .execute()
)
```

SQL:

```sql
INSERT INTO leads (phone, name, source) VALUES ('...', '...', 'whatsapp');
```

`.insert()` takes a dict (one row) or a list of dicts (multiple rows). The result has the

inserted rows in `.data`.

Example 5: Order + Limit

```python
res = (
    supabase.table("whatsapp_messages")
    .select("direction, body")
    .eq("lead_id", lead_id)
    .order("created_at", desc=True)
    .limit(limit)
    .execute()
)
```

SQL:

```sql
SELECT direction, body FROM whatsapp_messages
WHERE lead_id = '...'
ORDER BY created_at DESC
LIMIT 8;
```

- `.select("direction, body")` — multiple columns (comma-separated)
- `.order("created_at", desc=True)` — newest first
- `.limit(limit)` — cap the results

Common Filter Methods Cheat Sheet

Method	SQL equivalent
`.eq("col", val)`	`WHERE col = val`
`.neq("col", val)`	`WHERE col != val`
`.gt("col", val)`	`WHERE col > val`
`.gte("col", val)`	`WHERE col >= val`

`.lt("col", val)`	`WHERE col < val`
`.lte("col", val)`	`WHERE col <= val`
`.like("col", "%abc%")`	`WHERE col LIKE '%abc%'`
`.in_("col", [1,2,3])`	`WHERE col IN (1,2,3)`
`.is_("col", "null")`	`WHERE col IS NULL`

You can chain multiple filters — they're combined with `AND`:

```python
.eq("id", order_id).eq("status", "Pending PIN")
# WHERE id = '...' AND status = 'Pending PIN'
```

The Magic of Chained Filters (Race Condition Safety)

This is worth emphasizing. Look at this again:

```python
res = (
    supabase.table("orders")
    .update({"status": new_status})
    .eq("id", order_id)
    .eq("status", current_status)   # ← Only update if status is STILL the old one
    .execute()
)
if not res.data:
    return False
```

Why two `.eq()` calls? Because Supabase runs the **entire thing as one SQL statement**:

```sql
UPDATE orders SET status = 'Payment Failed'
WHERE id = '...' AND status = 'Pending PIN';
```

If two requests arrive at the same time:

- Request A runs first → status becomes "Payment Failed" → 1 row affected → `.data` has data
- Request B runs second → WHERE clause no longer matches → 0 rows affected → `.data` is empty → returns `False`

This is **atomic** — the database guarantees no race condition. Request B knows it lost the race and skips restoring stock.

Why Fluency Matters

Compare these three ways to fetch a product:

```python
# Raw SQL (verbose, error-prone)
supabase.execute("SELECT * FROM products WHERE id = %s", [product_id])

# ORM-ish builder (readable, chainable)
supabase.table("products").select("*").eq("id", product_id).execute()

# Direct HTTP to PostgREST (what Supabase actually does under the hood)
requests.get(f"https://xyz.supabase.co/rest/v1/products?id=eq.{product_id}",
             headers={"apikey": KEY})
```

The fluent API in the middle is what Supabase's Python client gives you — it's readable AND doesn't require learning SQL syntax.

🎭 Part 3: AI Prompt Engineering (The WhatsApp Bot)

This is arguably the most "creative" part of the code. Let me explain what's going on and why.

The Core Idea

You give the AI:

1. **A system prompt** — permanent instructions that say "here's who you are, here are the rules"
2. **A history of messages** — the conversation so far

The AI replies. That's it. But the *craft* is in how you design the prompt.

Dissecting the System Prompt

```python
TRIAGE_SYSTEM_PROMPT = """You are the friendly sales assistant for X-UK, a phone retail
er in Kenya.
Your job is to qualify WhatsApp leads and move them toward a purchase.

Rules:
- Ask ONE qualifying question at a time (budget, preferred brand/model, timeline, locat
ion).
- Keep replies short, warm and WhatsApp-friendly. No markdown headings.
- Never invent stock. If unsure, offer to connect the customer with a human.

You MUST return ONLY a JSON object (no other text) with this shape:
{
  "reply": "the message to send to the customer",
  "intent": "browsing|price|purchase|support|other",
  "interest": "specific phone or null",
  "budget": number or null,
  "timeline": "now|this_week|later|null",
  "location": "string or null",
  "score": 0,
  "stage": "new|warm|hot|cold"
}
`score` is an integer 0-100 estimating how ready the customer is to buy
(budget clarity + product specificity + urgency)."""
```

Let's break down each design decision:

1. Role definition

> "You are the friendly sales assistant for X-UK..."

Without this, the AI would respond generically. It might answer as a chatbot, or write essays, or explain things in ways that break the illusion.

2. Goal statement

> "Your job is to qualify WhatsApp leads and move them toward a purchase."

This tells the AI the *strategic* purpose. Without it, the AI might just answer questions passively. With it, the AI proactively asks for info that helps sales.

3. Behavioral rules

> "- Ask ONE qualifying question at a time..."

Why ONE? If the AI asks "What's your budget, what phone do you want, and when do you need it?", the customer feels interrogated and leaves. One question feels like a natural chat.

> "- Keep replies short, warm and WhatsApp-friendly. No markdown headings."

Why? Remember: this response will be sent as a WhatsApp text message. If the AI returns:

```
text

## Welcome!
Here is some information:

**Budget options:**
- Option 1: ...
```

...it looks terrible in WhatsApp. WhatsApp uses `*bold*` and `_italic_` (single asterisks), not markdown. So we explicitly forbid markdown.

> "- Never invent stock."

Why? LLMs *hallucinate* — they confidently make up facts. If the AI says "Yes, we have iPhone 15 in stock!" and you don't, you have an angry customer. This rule forces the AI

if none is in stock." and you don't, you have an angry customer. This rule forces the AI to say "let me connect you with a human" instead.

4. Output structure (the JSON schema)

> "You MUST return ONLY a JSON object (no other text)..."

This is the clever part. Normally, the AI would return human-readable text like:

> "Sure! The iPhone 15 costs 150,000 KES and I'd say you sound pretty interested in buying."

But we don't just want the reply — we want **structured metadata**:

- Did they mention a budget? (`budget`)
- What phone do they want? (`interest`)
- How ready are they to buy? (`score`)
- What's their urgency? (`timeline`)

So we force the AI to answer in JSON:

```
json

{
  "reply": "Sure! The iPhone 15 costs 150,000 KES. What's your budget like?",
  "intent": "price",
  "interest": "iPhone 15",
  "budget": null,
  "timeline": null,
  "location": null,
  "score": 45,
  "stage": "warm"
}
```

Now the bot can:

- Send just `reply` to the customer
- Save `score` , `interest` , `budget` , etc. to the `leads` table
- Trigger owner notifications when `score >= 70`

One AI call, two purposes (customer reply + structured data extraction).

5. The score definition

> " `score` is an integer 0-100 estimating how ready the customer is to buy (budget clarity + product specificity + urgency)."

Notice the AI isn't left to figure out "hot lead" on its own — we gave it **criteria**:

- **budget clarity** — "I have 100k" is +score
- **product specificity** — "iPhone 15 Pro Max 256GB" is +score vs "a phone"
- **urgency** — "I need it today" is +score

Without this, the AI would be arbitrary. With it, scores are consistent across conversations.

Why JSON Output is Tricky

Two problems arise with JSON output:

Problem 1: LLMs love markdown fences

Even when you say "return only JSON", the AI often returns:

````
text

```json
{
  "reply": "...",
  ...
}
```
````

Solution — `_parse_model_output` :

```
python

def _parse_model_output(raw: str):
```

```
    text = (raw or "").strip()
    if text.startswith("```"):
        text = text.strip("`")
        if text.lower().startswith("json"):
            text = text[4:].strip()
```

It strips the backticks and the word "json" if they're there.

```
python

    start, end = text.find("{"), text.rfind("}")
    if start != -1 and end != -1:
        try:
            data = json.loads(text[start:end + 1])
```

Then it finds the first `{` and last `}` and extracts everything between them. Even if the AI writes "Here is your JSON: {...} hope this helps!", the extraction still works.

```
python

            reply = str(data.get("reply") or "").strip()
            if reply:
                return reply, data
        except json.JSONDecodeError:
            pass
    return text, {}
```

If parsing fails entirely, it falls back to sending the raw text as the reply (with empty metadata). **Fail-safe design** — the customer always gets *something* back.

Problem 2: The AI can drift

Occasionally the AI ignores the "JSON only" instruction and just... talks. That's why the fallback exists. It's not that the prompt is wrong — it's that LLMs are probabilistic. You always design for the 95% case and handle the 5%.

Conversation History — Why It Matters

```
python
```

```
history = fetch_history(lead["id"])
messages = [{"role": "system", "content": TRIAGE_SYSTEM_PROMPT}]
for row in history:
    messages.append({
        "role": "user" if row.get("direction") == "inbound" else "assistant",
        "content": row.get("body", ""),
    })
```

LLMs are **stateless** — they don't remember anything between API calls. Every request must include the entire conversation.

So we rebuild it each time:

```
python

[
  {"role": "system",    "content": TRIAGE_SYSTEM_PROMPT},
  {"role": "user",      "content": "Hi, do you sell iPhones?"},
  {"role": "assistant", "content": "{\"reply\": \"Yes! Which model...\", ...}"},
  {"role": "user",      "content": "iPhone 15"},
  {"role": "assistant", "content": "..."},
  {"role": "user",      "content": "How much?"}   # ← new message
]
```

The AI sees the whole flow and replies in context.

Why `HISTORY_LIMIT = 8`?

You could send the entire history, but:

1. **Costs more tokens** — every request pays for input tokens
2. **Context limits** — models have a maximum token count
3. **Relevance** — the last 8 messages capture the current topic

Trade-off: 8 messages ≈ 4 back-and-forths, which covers most conversations. Older context is dropped (though the *lead metadata* is preserved in the DB).

A Subtle Detail: Roles

```python
"role": "user" if row.get("direction") == "inbound" else "assistant"
```

Note that **only the customer's messages are "user"** and **only our bot's replies are "assistant"**. The store owner's messages don't exist yet. This matters — the AI sees itself as the assistant in this conversation, not as some third party.

Temperature Tuning

```python
raw = await chat_completion(messages, temperature=0.4, max_tokens=500)
```

Recall `temperature`:

- **0** = deterministic, always picks the most likely next token
- **1** = creative, lots of randomness
- **0.7** = default in `ai.py`

For the sales bot, `0.4` is deliberate:

- **Too low (0.0)**: overly rigid, robotic responses
- **Too high (0.8+)**: creative, but might break JSON format or hallucinate

`0.4` = friendly but consistent. Especially important when you need valid JSON every time.

The Full Flow Diagram

```text
Customer: "Hi, do you have iPhone 15?"
       ↓
[1] verify_signature → is this really from Meta?
       ↓
[2] extract_inbound_message → {"from": "+254...", "text": "Hi..."}
```

```
       ↓
[3] get_or_create_lead → find/create in DB
       ↓
[4] log_message → save "Hi..." as inbound
       ↓
[5] fetch_history → last 8 messages
       ↓
[6] Build messages array:
    [system_prompt, ..., "Hi, do you have iPhone 15?"]
       ↓
[7] chat_completion → OpenRouter → Llama 3.1
       ↓
[8] AI returns JSON:
    {"reply": "Yes! Which model?", "interest": "iPhone", "score": 30, ...}
       ↓
[9] _parse_model_output → extract reply + metadata
       ↓
[10] log_message → save reply as outbound
       ↓
[11] _apply_meta → update lead: score=30, status="cold"
       ↓
[12] send_whatsapp_message → Meta Graph API → customer's phone
       ↓
[13] If status changed to "hot" → _notify_owner
```

🎯 Summary: What You Learned

HMAC Signatures

- **Purpose**: Prove a webhook came from a trusted source
- **Mechanism**: Shared secret + hash function = unforgeable signature
- **Gotcha**: Use `compare_digest` to prevent timing attacks

Supabase Queries

- **Fluent API**: `.table().select().eq().execute()`
- **SQL mapping**: Every method maps to a SQL clause

- **The atomic trick**: `.eq("status", old_status)` in an UPDATE prevents race conditions

AI Prompt Engineering

- **System prompt**: defines role, rules, output format
- **Structured output**: force JSON to get both reply AND metadata in one call
- **Defensive parsing**: strip fences, extract `{...}`, fall back to raw text
- **Statelessness**: rebuild history on every call
- **Temperature**: lower for structured/consistent output

Want me to go deeper on any of these — like **how OpenRouter's API actually works**, **how FastAPI runs the webhook**, or **how to design a similar system from scratch**?